Q: Any questions or comments?

Lecture 17: Distributed Algorithms

COMP2014 Distributed Systems

Chris Greenhalgh

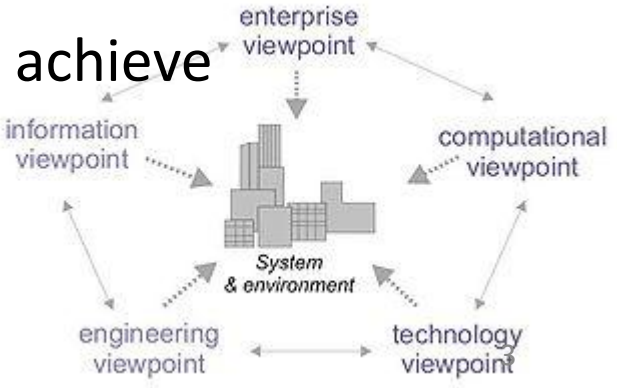
School of Computer Science

Contents

- Distributed Algorithms
- Interaction Models
  - Synchronous & Asynchronous Distributed Systems
- Examples of distributed algorithms
  - Logical Time
  - Distributed Hash Table

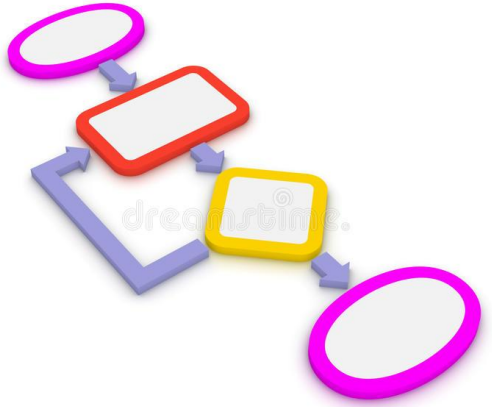
Recap: Fundamental models

- …contain only the essential elements to allow us to reason about some aspect of the system.
- In particular we may model:
  - **Failure**: faults that can occur, their potential effects, and how faults can be tolerated while continuing to run correctly (see later lectures)
  - **Security**: forms of attack, analysis of threats and how to resist them (see security lecture)
  - **Interaction**: computational processes passing messages to achieve communication (information flow) and coordination (synchronisation and ordering) (this lecture!)



Distributed Algorithms (Section)

Distributed algorithms



- An **algorithm** describes a sequence of steps to be taken to perform a particular operation
  - It specifies the variables (state) to be maintained and the instructions (steps) to be performed
- A **distributed algorithm** describes the steps to be taken by *each process* in the distributed system, including sending/receiving *messages* to achieve a common goal
  - A *distributed* system is made up of multiple processes (or nodes), each with its own *separate* state and instructions, which can only exchange *messages* to transfer information and coordinate

Q: can a distributed algorithm have different instructions on different nodes?

Thinking about distributed algorithms

Q: Can you name any (uses of) distributed algorithms?

- A distributed algorithm is intended to achieve one or more **goals**/outcomes
  - May be expressed as logical propositions or predicates
  - E.g. routing packets & building routing tables, achieving consensus, electing a leader.
- A **correct** algorithm will satisfy those goals – hopefully **provably** – provided its assumptions are met
  - But if the **constraints** are not met then the algorithm is likely to fail
  - E.g. a synchronous algorithm will not work reliably on an asynchronous system
- One distributed algorithm may provide a basis on which to build another
  - E.g. an algorithm for achieving reliable group communication over unreliable communication channels may be a useful foundation for many other algorithms
  - Typically the “lower” algorithm will make weaker assumptions about the system it is running on, and will allow the algorithm that uses it to make stronger assumptions

Interaction Models (Section)

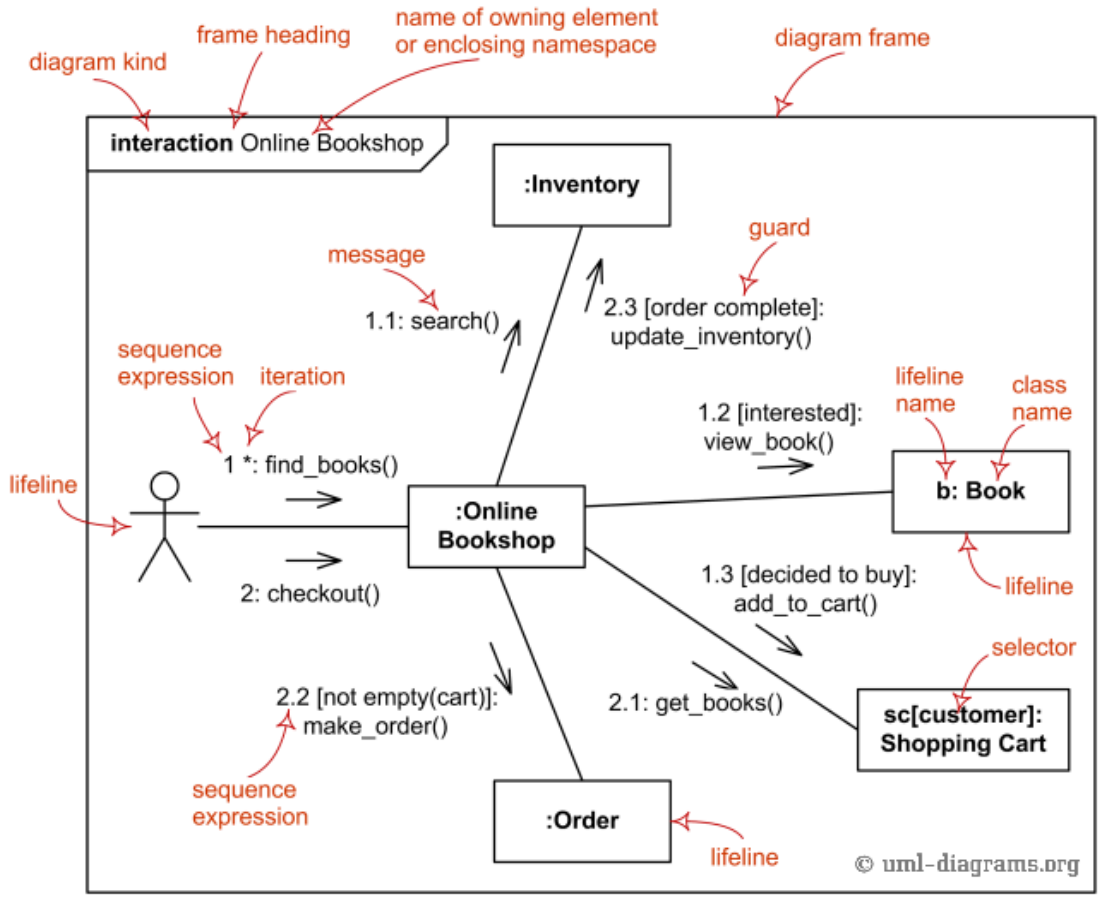
Interaction models & assumptions

Q: What does a non-distributed algorithm (usually) assume?

- So what **assumptions** can we make in an interaction model or distributed algorithm? e.g.:
  - What underlying **communication paradigm** is available, e.g. one-to-one message passing, reliable group communication, …
  - Whether the **entities** (e.g. **processes**) involved are **fixed** or varying (**dynamic**)
  - Whether communication is **reliable**, its **bandwidth** and **latency**
  - Whether each node has a reliable **clock**
  - Whether processes can **fail**, and if so how
  - Whether processes and communication channels can be **trusted** or may be **malicious**, i.e. **security**

E.g. OO Communication Diagram or non-distributed algorithm

- We (usually) assume:
  - Communication paradigm is (generally) synchronous method invocation
  - Entities are objects; a particular known set/arrangement of objects
  - Communication is reliable and fast
  - Often on the same node, so have access to the same clock
  - Individual objects do not fail independently (but may signal exceptions as messages)
  - Method invocation is trusted and secure
- So what about a distributed system??



Synchronous distributed systems

- A "**synchronous**" distributed system is one in which:
  - Time to execute each step of a process has known upper and lower bounds
  - Each message transmitted over a channel is received within a known bounded time
  - Each process has a clock whose drift rate from real time has a known bound (see later slide)
- Useful (or essential) to build dependable realtime systems
  - E.g. fly-by-wire aircraft, modern car
  - Requires a realtime operating system as well as specialised (reliable) communication links!

Asynchronous distributed systems

But most distributed systems are not synchronous…

- An **asynchronous** distributed system (e.g. the Internet) has no bounds on (one or more of):
  - Process execution speeds
  - Message transmission delays
  - Clock drift rates
- For example,
  - a busy web server can take a very long time to handle a request,
  - an email can take days to arrive,
  - a server's clock can differ by any amount from the real time

Synchronous vs Asynchronous Distributed Systems

- Modelling interaction in a synchronous system can be useful, and simpler than the alternative
  - E.g. timeouts can determine that a message was never sent or that a step was never performed
- Some problem are impossible to solve in an asynchronous system,
  - but some can be solved,
  - and such systems are still very useful (and unavoidable…)
    - E.g. a web browser cannot guarantee a timely response, but it can allow the user to do other things while they wait.

Computer clocks and timing events

- Most computers have an internal **clock** which local processes can use to get the current **time**
- But two processes in a distributed system reading their clocks at the same moment can get *different* time values
- Each clock will **drift** from perfect time
  - *Clock drift rate* is how quickly the clock drifts (a fraction)
- Clocks can be corrected to some extent by:
  - Using a GPS receiver on each computer where available/cost-effective => ~ 1 microsecond (0.000001 second) accuracy
  - Sending messages to other processes => accuracy depends on communication latency

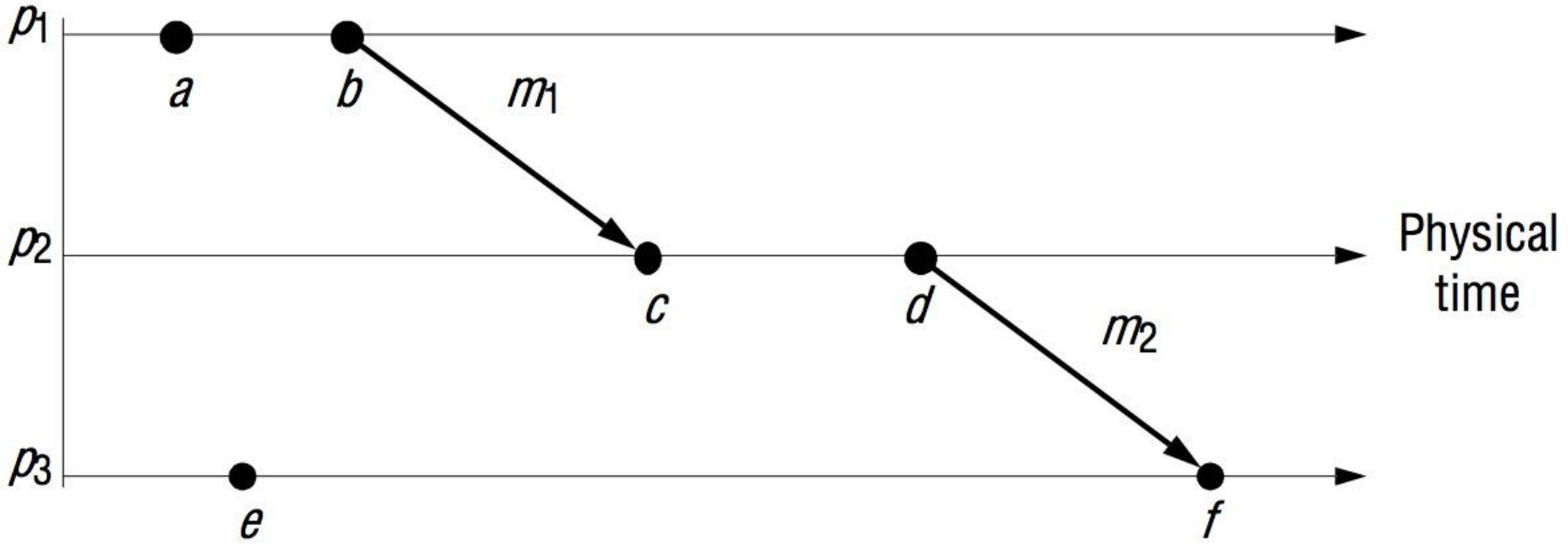
Example: Logical Time

Note, the precise details of Lamport's Algorithm are **not examinable**, but a general understanding distributed algorithms and event ordering (as in group communication) is expected

Logical time

- In an **asynchronous** system
  - host clocks are not synchronised
  - and so cannot provide a definite ordering of events happening at different hosts
- How can we ensure that every host can order events in a way that is **consistent**?
  - (i.e. causal order - see Indirect Communications – group comms)
  - E.g. …

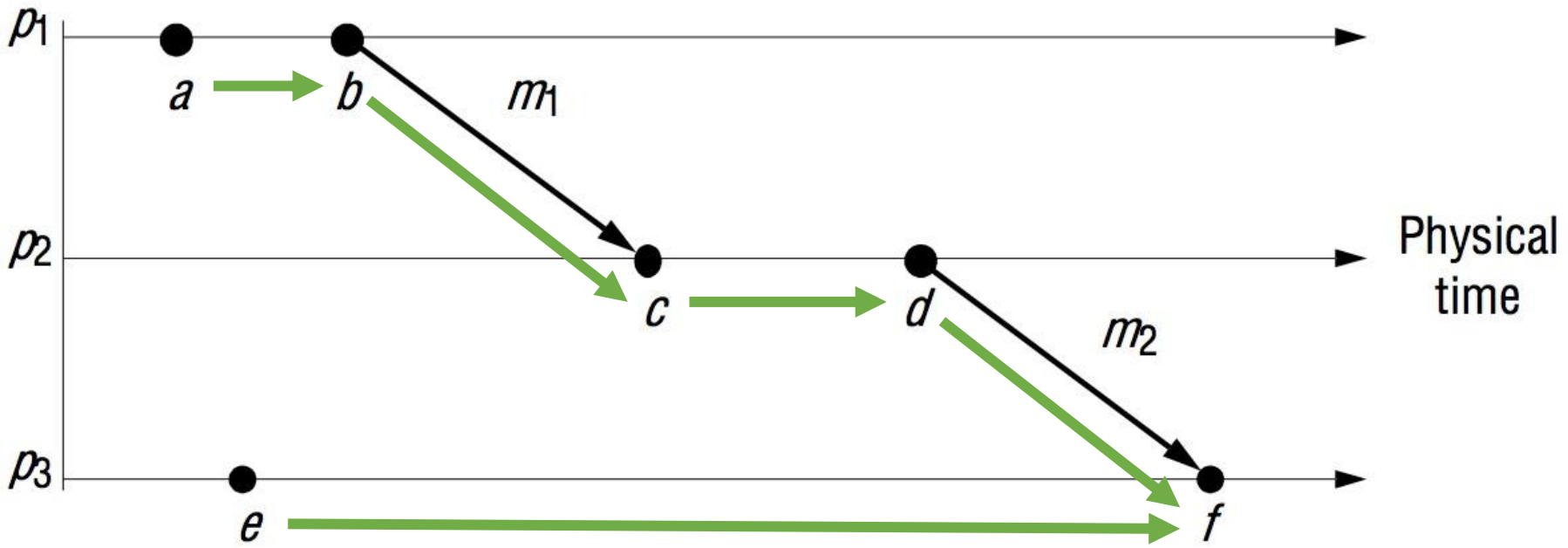
Figure 14.5
Events occurring at three processes



First define the goal…

- We want to preserve the **logical** relationships between events
  - In particular (possible) dependencies between events
- Represented as the "**happened before**" relation, "→"
  - E.g. "*a* → *b*" means event *a* happened before event *b*
  - If *c* happens before *d* in the *same process* then c → d
  - *Sending* a message always happens before *receiving* that message
    - i.e. "send(*m*) → recv(*m*)"

Marked up with “happened before”…



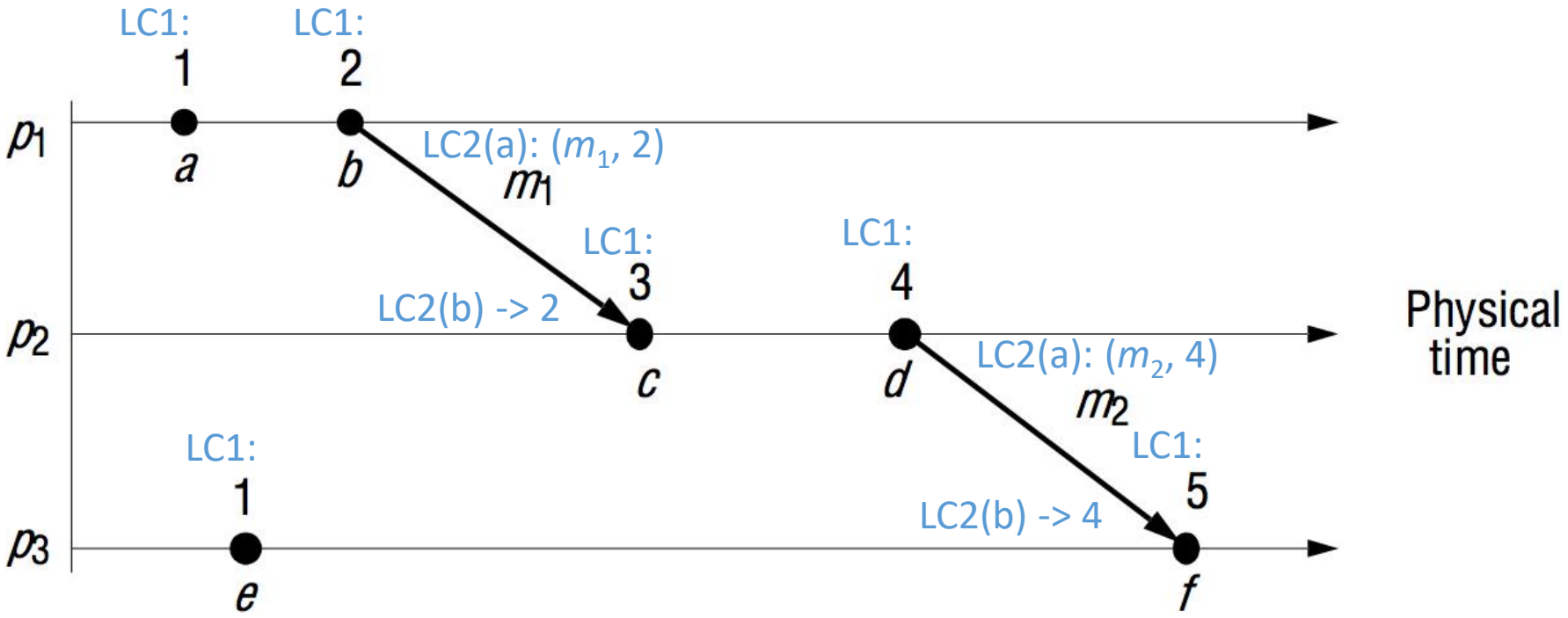
Note

- *Happened before* is **transitive**
  - i.e. *a* → *b* and *b* → *c* => *a* → *c*
- Happened before is a **causal** order
  - i.e. if information from *a* could have been available (through messages or local state) when *b* was done, then *a* → *b*
- Happened before is not a **total** order
  - E.g. *a* doesn't happen before *e* and neither does *e* happen before *a*,
  - so some processes may see *a*,*e* and others may see *e*,*a*
- (See Indirect Communications – group comms)

Lamport's logical clock algorithm
Lamport [1978]

- Each process p_i maintains a **logical clock** L_i which is used to assign Lamport timestamps to each event
  - $L_i(e)$ is the timestamp of event e at process i
  - $L(e)$ is the timestamp of event e at the process it occurred at
- LC1: L_i is incremented before each event at p_i
- LC2(a): when p_i sends a message m it piggybacks the value $t = L_i$
- LC2(b): on receiving (m,t) at p_j, do $L_j = max(L_j, t)$ then LC1 then timestamp *receive*(m)

Figure 14.6 Lamport timestamps for the events shown in Figure 14.5



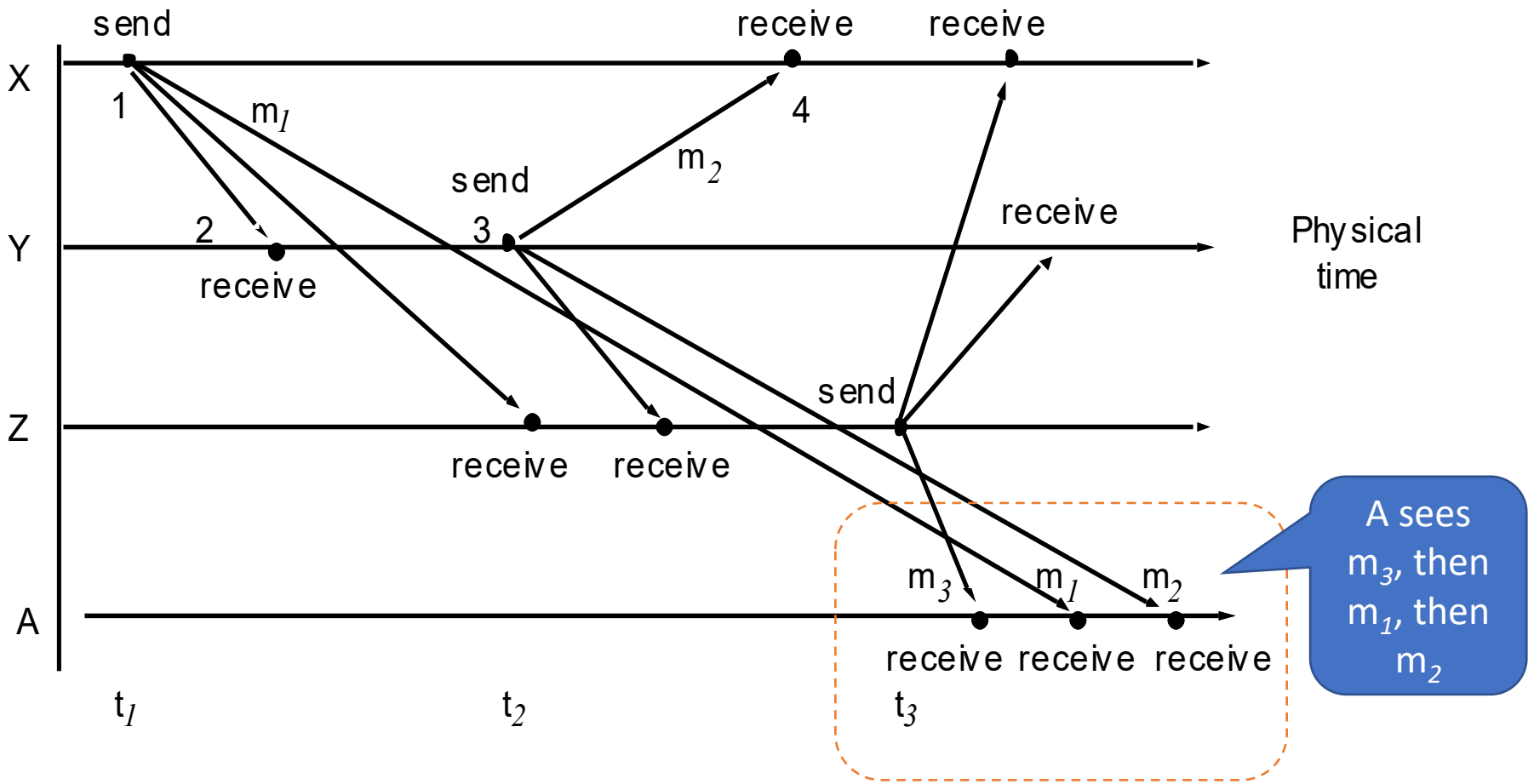
Notes

- One can easily show that if $a \rightarrow b$ then $L(a) < L(b)$
  - However $L(a) < L(b)$ does not imply that $a \rightarrow b$
    - E.g. $L(e) = 1$ and $L(b) = 2$ but it is not the case that $e \rightarrow b$
  - So Lamport's algorithm is sufficient but not necessary to enforce *happened before*, i.e. causal order
- Note: this algorithm **assumes**
  - Processes can be trusted
  - Messages may be lost but will not be corrupted
  - Processes may crash but not give arbitrary errors
    - In each case you can construct counter-examples that fail to respect happened before

Example: message (re)ordering

- Consider 4 email users X, Y, Z, A on a mailing list:
  - X sends a message (m_1) with the subject “Meeting”
  - Y and then Z reply sending messages (m_2, m_3, respectively) with the subject “Re: Meeting”
- Because of independent delays in communication the actual order of message deliveries may be:

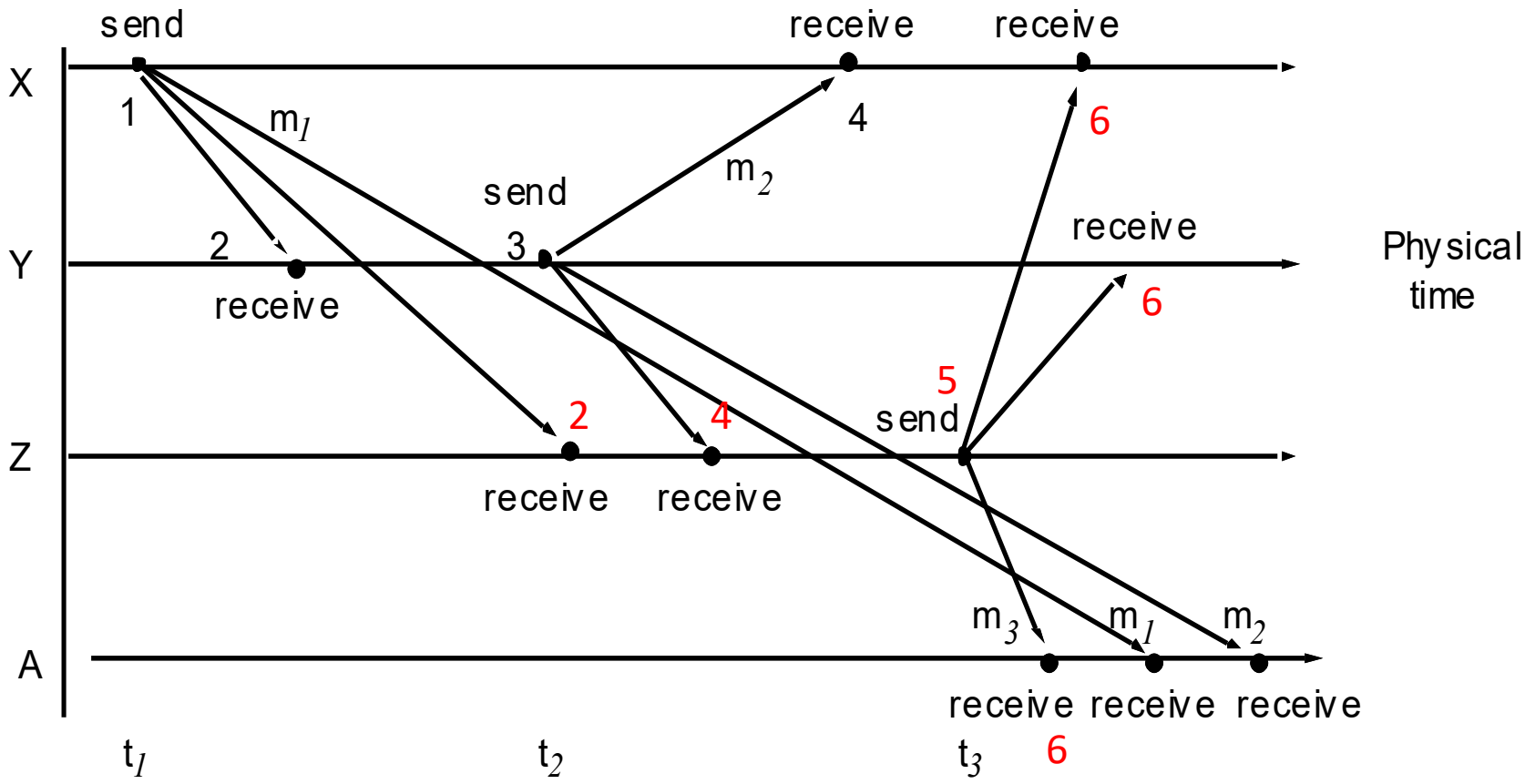
Figure 2.13: Real-time ordering of events – received out of order



Message (re)ordering notes

- If the clocks could be synchronised then each message could include the time when it was sent (t1, t2, t3), which could be used to put the messages in order (t1<t2<t3)
- Alternatively logical time can preserve the order without clocks
  - the numbers on the next figure show logical times for some of the events which could be used to sort them in order at A…

Figure 2.13: Real-time ordering of events – with logical time



Sent: 5 1 3

Sort => m_1 (1), m_2 (3), m_3 (5)

Example: Routing in Distributed Hash Tables

Note, the details of DHT implementation and routing are **not examinable**, but a general understanding distributed algorithms and the basic idea of overlay networks is expected

Recap: Distributed hash tables (DHT)

- An overlay network that maintains a set of **values**
  - E.g. files
- Each value associated with a unique **key**
  - E.g. a **secure hash** of an arbitrary-length key (or the value)
  - i.e. it is a hash table, but distributed ☺
- Peers can
  - **Add** a key/value to DHT
  - **Remove** a key/value from the DHT
  - **Get** a value from the DHT given its key

DHT / Pastry Strategy

In the Pastry DHT

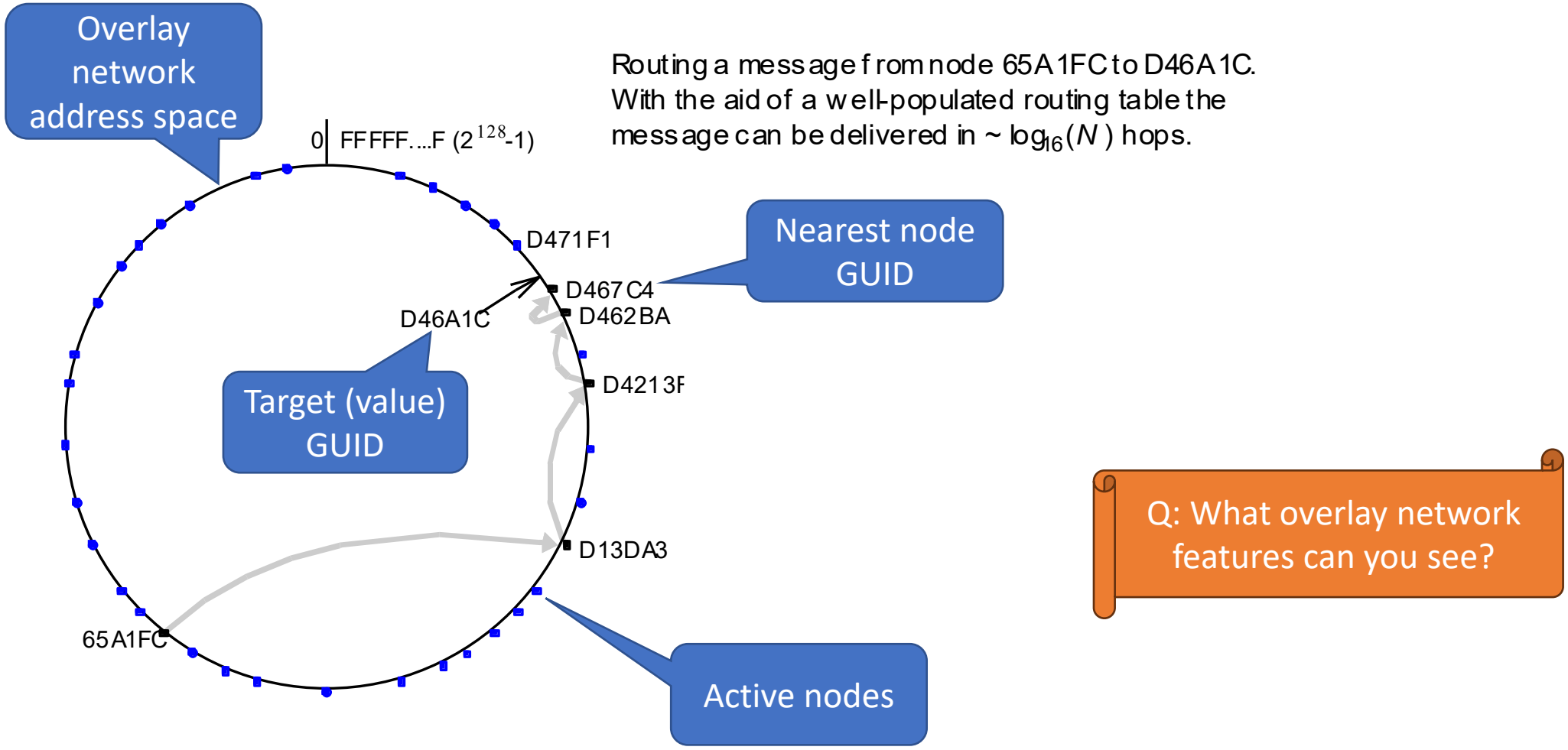
- each **node** has a GUID (Globally Unique IDentifier)
- Each **value**/file has a GUID as its **key**
- Values are stored at the node whose GUID is **closest** to the value's GUID
  - Copies can also be kept in "neighbouring" nodes for reliability/performance
- The nodes organise themselves into an overlay network that is a **ring** sorted in order of GUID

Pastry request routing

- i.e. **routing** requests to add/remove/get a specified value by its GUID
  - Needs to get to the node with **closest GUID**
- So a request can be routed in the overlay network simply by sending it to the neighbour in the direction with the **closer** GUID
  - Each node knows it neighbours so it knows whether it is current the closest node
  - Each hop will get closer
  - So eventually it will reach the right node
    - Provided nodes are not arriving/leaving “too” quickly

Figure 10.8: Pastry routing example

Based on Rowstron and Druschel [2001]



Pastry Notes

- Actually nodes build up bigger **routing tables** (not just immediate neighbours) that allow them to skip blocks of neighbours to speed things up
  - Can show that this is still correct
- We are **assuming** (i.e. requiring)
  - Some other part of the system maintains the underlying communication **ring**
    - Including inserting new nodes and removing leaving/failed nodes
  - Nodes can be **trusted** to behave correctly

Distributed Algorithms Summary

- A **distributed algorithm** describes the steps to be taken by *each process* in the distributed system, including sending/receiving messages
  - A distributed algorithm is intended to achieve one or more **goals**/outcomes
  - A distributed algorithm will depend on a particular set of **assumptions** about the system that is executing it, in particular about **interaction**, **failure** & **security**
  - A **correct** algorithm will satisfy those goals – hopefully **provably** – provided its assumptions are met
- For example, in a distributed system events can be delivered in arbitrary order at different recipients, but **logical time** can be used to re-order them correctly
- One distributed algorithm may provide a basis on which to build another

Interaction Models Summary

- Interaction models focus on processes, state and message exchange
- Interaction models make **assumptions** about the nature of the underlying system, e.g. **communication**, **failure** & **security**
- A **synchronous distributed system** has known upper bounds on: execution time; message transmission delay; clock drift rate
  - An **asynchronous distribute system** doesn't
- Clocks in a distributed system are subject to **drift**: there is no perfect global clock.

Next

- Lab 8: multicast
- Lecture 18: Failure & reliable communication
- Remaining lecture(s):
  - Lecture 19: (More) Data & Failure
  - Q&A/revision x 1-2